MID-SEM REVISION | ALL 6 FILES MERGED

System Programming

Study Guide

Everything for your exam in one place.
Simple words, exact answers, colour-coded diagrams.

15 topics

9 diagrams

14 exam answers

HOW TO SPOT WHAT MATTERS

- Highlighted phrases = keywords the examiner looks for
- WRITE THIS boxes = the exact answer to put on paper
- Tables = compare things quickly

Contents

Tap any row to jump straight to that topic.

01	What is System Programming?	FOUNDATIONS
02	Program, Process & Thread	FOUNDATIONS
03	Computer System Architecture	HARDWARE
04	CPU Components	HARDWARE
05	The Instruction Cycle	HARDWARE
06	Memory Hierarchy	HARDWARE
07	Process Lifecycle: With OS vs Without OS	HARDWARE + OS
08	Input / Output System	HARDWARE
09	System Software vs Application Software	SOFTWARE
10	Types of System Programs	SOFTWARE
11	Levels of Programming Languages	SOFTWARE
12	Request-Response, Client-Server & 3-Tier	WEB SYSTEMS
13	Types of Requirements	SOFTWARE ENGINEERING
14	Types of Environments	SOFTWARE ENGINEERING
15	Cheat Sheet: every definition in one place	LAST-MINUTE REVIEW

HOW TO USE THIS GUIDE

1. Read the **WRITE THIS** box in each topic. It is the exact answer to write in the exam, in simple words.
2. The **highlighted phrases** are the keywords a marker looks for. Learn those first.
3. Look at the diagram to actually understand it, then use the table to compare things.
4. The night before the exam: only read the **Cheat Sheet** on the last page.

FOUNDATIONS

01 What is System Programming?

System programming means writing code that sits close to the hardware and the OS, instead of code that solves a user-facing problem directly. Its job is to **manage resources** (CPU time, memory, I/O devices) and hand those resources to application software through a clean interface.

WRITE THIS **"What is System Programming?"**

System programming means writing programs that work closely with the **operating system and the hardware**, instead of solving a task for the end user directly. Its main job is to **manage the computer's resources: the CPU, the memory, and the I/O devices** and provide these as services to application software. Common examples of system programs are **compilers, linkers, loaders, device drivers, and the operating system itself**. We study it to understand how a program really interacts with hardware and the OS, and to build skills in memory management, process handling, and automation.



Fig 1: where system programming sits in the stack

Why it is on the syllabus

- Shows you how a program actually talks to the OS and hardware, instead of treating that as a black box.
- Builds your understanding of **memory, processes and system-level behaviour**, the stuff that explains why code behaves the way it does.
- Develops **scripting and automation** skill for real tasks (the practical payoff of the theory).

EXAM ONE-LINER

System programming = programming close to the OS/hardware, **managing CPU, memory and I/O**, and providing services to application software. Examples: compilers, linkers, loaders, device drivers, the OS itself.

02

FOUNDATIONS

Program, Process & Thread

These three words get mixed up constantly. The exam loves testing exactly this difference.

WRITE THIS **"Differentiate Program, Process and Thread"**

A **program** is simply a set of instructions written in a programming language and stored in secondary memory (disk); it is **not being executed**, for example a file like hello.c. A **process** is a **program that is currently in execution**: it is loaded into main memory and is given its own CPU time, memory, and I/O resources, along with a unique Process ID (PID). A **thread** is a **lightweight unit of execution inside a process**; it shares the code, data, and heap of its parent process but has its own stack, and is mainly used to run multiple tasks in parallel, like multiple tabs open in the same browser.

STATIC

Program

A set of instructions in a programming language, sitting on disk. Not running. E.g. a hello.c file.

RUNNING

Process

A program in execution. Has its own CPU time, memory, I/O and a unique PID. E.g. running ./a.out.

INSIDE A PROCESS

Thread

A lightweight execution unit inside a process, used for parallel work. E.g. each tab in a browser.

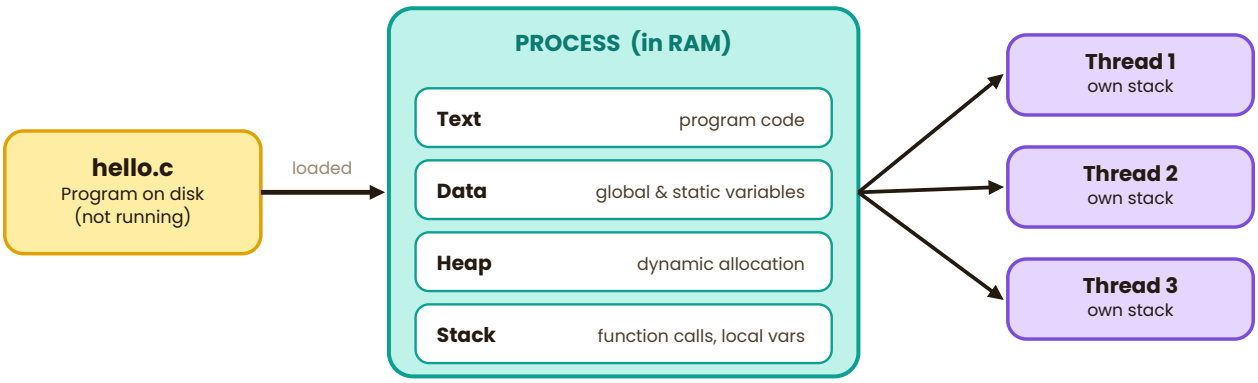


Fig 2: a program becomes a process; a process can spawn threads (they share Code, Data and Heap, but each has its own Stack)

TERM	STATE	WHERE IT LIVES	NEEDS
Program	Not executing	Secondary memory (disk)	Nothing. It is just stored instructions
Process	Executing	Main memory (RAM)	CPU, memory, I/O resources + a PID
Thread	Executing (inside a process)	Shares its parent process's memory	Its own stack and registers only

EASY WAY TO REMEMBER

Program = a recipe written on paper. Process = you cooking that recipe right now. Thread = several hands working in the same kitchen on the same dish.

03

HARDWARE

Computer System Architecture

This is the conceptual design of a computer: how hardware and software interact to actually run a program. Four pieces matter: the **CPU**, **memory**, **I/O devices**, and the **system bus** that connects them.

WRITE THIS **"Explain Computer System Architecture"**

Computer system architecture is the **conceptual design and basic operating structure** of a computer. It defines how the hardware and software interact so that programs can run. Its main components are the **CPU**, which controls operations and executes instructions; **memory**, which stores instructions and data in a hierarchy from registers to cache to RAM to disk; **I/O devices**, which handle input like the keyboard and mouse and output like the monitor and printer; and the **system bus**, which physically connects the CPU, memory, and I/O devices using data, address, and control lines.

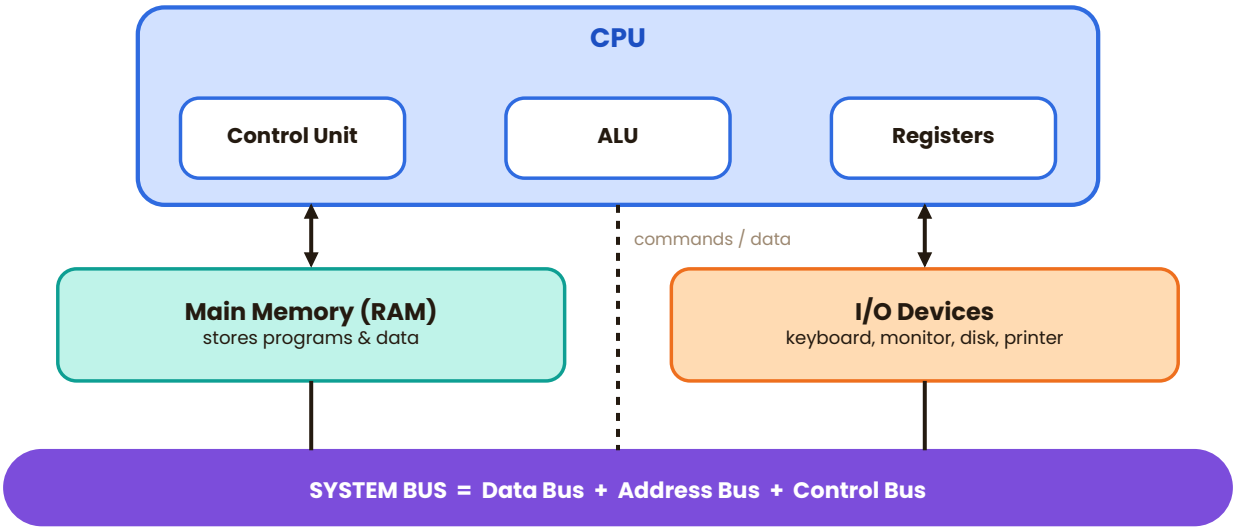


Fig 3: CPU, memory and I/O all talk through the system bus

CPU
Controls operations, executes instructions.

Memory
Stores instructions and data: Registers, Cache, RAM, Disk.

I/O Devices
Input (keyboard, mouse) and output (monitor, printer).

EXAM ONE-LINER

System Bus = **Data Bus** (carries the actual data) + **Address Bus** (says which memory location) + **Control Bus** (carries read / write / interrupt signals).

04 HARDWARE

CPU Components

Zooming into the CPU box from Fig 3, these four things are what is actually inside it.

WRITE THIS **"Explain the components of a CPU"**

The CPU is made up of four main parts. **Registers** are small, very fast storage locations inside the CPU that temporarily hold instructions, addresses, and data. Examples are the Program Counter (PC), Instruction Register (IR), and Accumulator (ACC). The **Arithmetic Logic Unit (ALU)** performs arithmetic operations like addition and subtraction, and logical operations like AND, OR, and comparison. The **Control Unit (CU)** directs the entire process: it decodes instructions and manages the ALU and registers. Finally, the **buses** (data bus, address bus, and control bus) are the communication pathways that connect the CPU to memory and I/O devices on the motherboard.

COMPONENT	ROLE	NATURE / LOCATION
Registers	Temporary storage for instructions, addresses, data: PC (Program Counter), IR (Instruction Register), ACC (Accumulator)	Hardware; tiny memory inside the CPU core
ALU	Arithmetic (add, subtract, multiply) and logical (AND, OR, NOT, compare) operations, working directly on register data	Hardware circuit inside the CPU core
Control Unit	Directs flow of data and instructions, decodes instructions, manages the ALU and registers	Hardware + microprograms, inside the CPU core
Buses	Data Bus (actual data), Address Bus (memory location), Control Bus (read / write / interrupt signals)	Hardware wires on the motherboard, connecting CPU, RAM and I/O

EASY WAY TO REMEMBER

PC = address of the NEXT instruction. IR = the instruction being worked on RIGHT NOW. ACC = holds data / results.

05

HARDWARE

The Instruction Cycle

The CPU does not do anything "in one go". Every single instruction goes through the same repeating 4-step loop.

WRITE THIS **"Explain the Instruction Cycle"**

The CPU executes every single instruction using a repeating four-step process called the **instruction cycle**. First, in the **Fetch** step, the instruction is retrieved from memory and placed into the Instruction Register. Next, in the **Decode** step, the Control Unit interprets what the instruction means and prepares the required control signals. Then, in the **Execute** step, the ALU or the control circuits actually carry out the operation. Finally, in the **Store (write-back)** step, the result is saved back into a register or memory if needed. After each cycle, the **Program Counter** updates to point to the next instruction, and this cycle keeps repeating until the program stops.

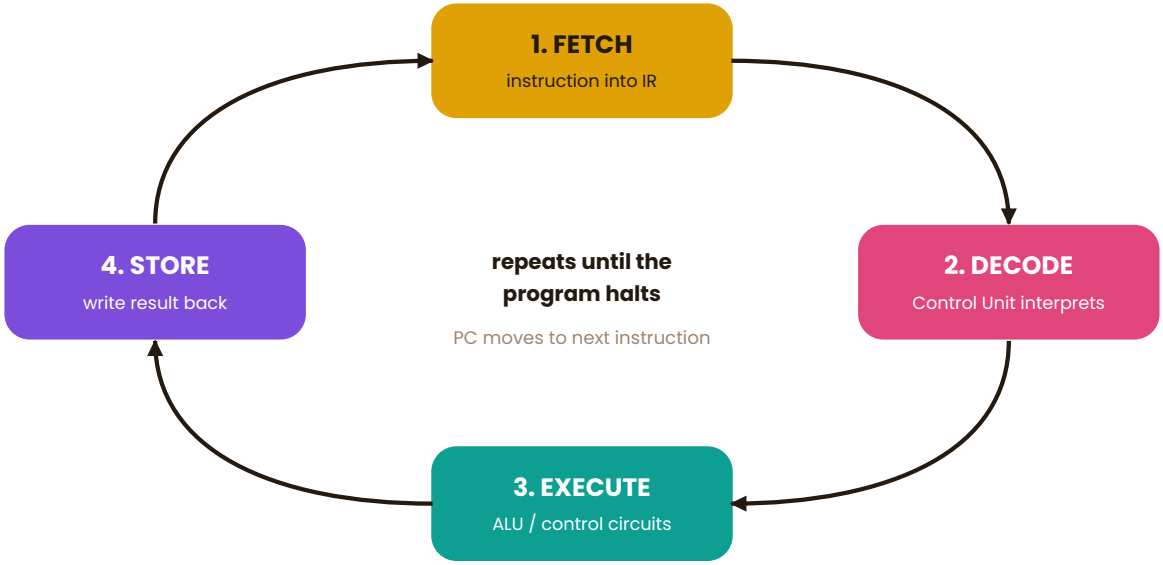


Fig 4: Fetch, Decode, Execute, Store, then the Program Counter moves on

- **Fetch**: the instruction is fetched from memory into the Instruction Register (IR).
- **Decode**: the Control Unit interprets the instruction and prepares the signals it needs.
- **Execute**: the ALU (or control circuits) performs the actual operation.
- **Store / write-back**: the result is written to a register or memory, if needed.

EASY WAY TO REMEMBER

Say it as one word: F-D-E-S. Fetch, Decode, Execute, Store.

06 HARDWARE

Memory Hierarchy

Not all memory is equal. There is a clear trade-off: the faster it is, the more expensive and smaller it is.

WRITE THIS **"Explain Memory Hierarchy"**

Computer memory is arranged in a **hierarchy** based on speed, size, and cost. At the top are **registers**, which are the fastest and smallest, located inside the CPU itself. Below that is **cache memory**, which is very fast but expensive, and sits between the CPU and RAM. Next is **RAM (main memory)**, which is fast and holds the programs that are currently running. Below RAM is **secondary storage** like hard disks and SSDs, which is much larger but slower. At the bottom is **tertiary storage**, such as CDs, tapes, or cloud storage, which has the largest capacity but is the slowest. The overall trade-off is: **as speed increases, cost increases and size decreases**.

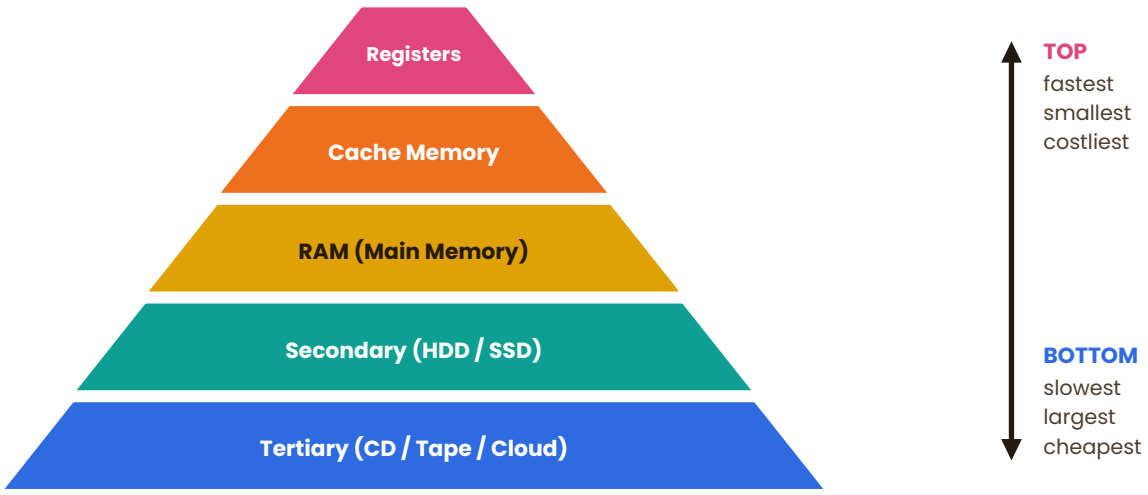


Fig 5: fastest and smallest at the top, largest and slowest at the bottom

LEVEL	SPEED	SIZE / COST	HOLDS
Registers	Fastest	Smallest, most expensive	Data the CPU is using right now
Cache	Very fast	Small, expensive	Temporary buffer between CPU and RAM
RAM	Fast	Medium	Currently running programs / data
Secondary (HDD / SSD)	Slower	Large	Files, installed programs
Tertiary (CD / Tape / Cloud)	Slowest	Very large, cheapest	Long-term archive / backup

07

HARDWARE + OS

Process Lifecycle: With OS vs Without OS

A classic exam contrast: what actually changes if you remove the operating system and run "bare metal".

WRITE THIS **"Process Lifecycle with OS vs without OS"**

When an operating system is present, a program on disk is first **loaded into memory by the OS loader**, and split into text, data, heap, and stack segments. The OS then creates a **Process Control Block (PCB)** to track the process's ID, state, register values, memory map, and I/O status. A **scheduler** decides which process gets the CPU next, using **context switching** to save and restore registers. During execution, the CPU's registers, control unit, ALU, and buses work as usual, and if the process needs I/O, the OS puts it in a waiting state. When the process finishes, the OS frees its memory and marks it as terminated. **Without an operating system**, the program is loaded directly into memory, for example by a bootloader, and **only one program can run at a time** with no scheduling. The CPU executes it directly with full control and no context switching, **the program itself must handle any I/O** using device registers, and when it finishes, the CPU simply halts or waits to be reset.

STAGE	WITH AN OS	WITHOUT AN OS (BARE METAL)
Loading	OS loader loads the program from disk into memory, split into Text / Data / Heap / Stack segments	Program is loaded directly into memory, e.g. by a bootloader
Bookkeeping	OS creates a Process Control Block (PCB): PID, state, register snapshot, memory map, I/O status	None. No PCB, no tracking
Scheduling	Scheduler decides which process gets the CPU; context switches save / restore registers and PC	None. Only one program ever runs, with full CPU control
Execution	CU fetches, ALU executes, buses move data, all under OS control	CU fetches / decodes directly from memory; no context switching
I/O	OS issues the I/O command via the control bus and puts the process in a waiting state	Program itself handles I/O directly (device registers / memory-mapped I/O)
Termination	OS deallocates memory, marks the PCB "terminated", removes it from the queue	CPU simply halts or waits for a reset

EXAM ONE-LINER

The OS's whole job in this comparison is **bookkeeping + scheduling**. The CPU's fetch-decode-execute cycle underneath is the same either way.

08

HARDWARE

Input / Output System

Input devices (keyboard, mouse, scanner) and output devices (monitor, printer, speaker) both talk to the CPU through an **I/O Controller**. How that conversation happens comes down to one of three techniques.

WRITE THIS **"Explain I/O techniques"**

Input devices like the keyboard, mouse, and scanner provide data to the computer, while output devices like the monitor, printer, and speaker display the results. An **I/O controller** manages the transfer of data between the CPU and these devices. There are three techniques used for this: in **Programmed I/O**, the CPU directly manages the transfer and has to wait until it is done; in **Interrupt-Driven I/O**, the device sends an interrupt signal to the CPU only when it is ready, so the CPU is free to do other work in the meantime; and in **Direct Memory Access (DMA)**, the data transfer bypasses the CPU entirely, which makes it the most efficient of the three.

TECHNIQUE	HOW IT WORKS	CPU COST
Programmed I/O	CPU directly manages the I/O operation itself	High. CPU busy-waits until the device is done
Interrupt-Driven I/O	Device interrupts the CPU only once it is ready	Lower. CPU is free to do other work while waiting
DMA	Data transfers bypass the CPU entirely	Lowest. CPU is not involved in moving the data at all

EASY WAY TO REMEMBER

Programmed = the CPU stands there waiting. Interrupt = the device taps the CPU on the shoulder when ready. DMA = the device talks to memory directly, CPU not needed.

09

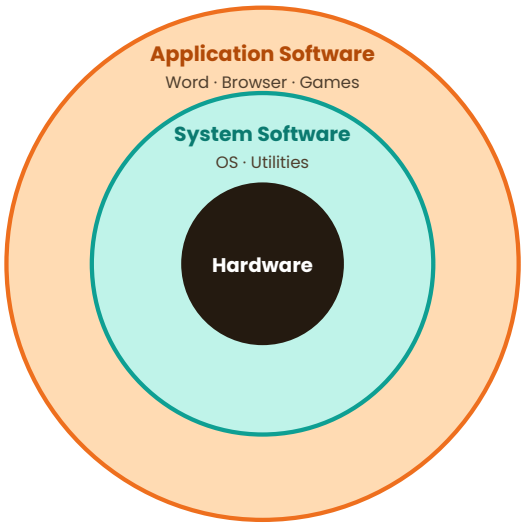
SOFTWARE

System Software vs Application Software

You interact with **application software** directly. That layer talks to **system software** (OS + utilities), which in turn talks to the **hardware**.

WRITE THIS **"Differentiate System Software and Application Software"**

System software is the software that acts as an **interface between the application software and the computer hardware**; it is close to the hardware, usually **runs in the background**, and is mostly invisible to the user. Examples are the operating system, compilers, and device drivers. Application software, on the other hand, is designed to help the user perform a specific task; it **runs on top of system software**, has a proper user interface, and is easy to install and use. Examples are word processors, web browsers, and games. System software is usually written in **low-level languages** for hardware compatibility and can run independently, whereas application software is written in **high-level languages** and **cannot run without system software** present underneath it.



Application software
what YOU use: Word, browser, games

System software
OS + utilities; sits between apps and hardware

Hardware
CPU, disk, mouse, printer

Fig 6: application software wraps system software, which wraps hardware

	SYSTEM SOFTWARE	APPLICATION SOFTWARE
Definition	The interface between application software and the system	Runs per user request, on the platform system software provides
Dev language	Usually low-level, for hardware compatibility	Usually high-level, for a specific purpose
Usage	Operates the computer hardware	Used by the end user for a specific task
Installation	Installed when the OS is installed	Installed as needed by the user
User interaction	Little to none. No user interface	Direct. Has a user interface
Dependency	Runs independently; provides the platform	Can't run without system software present
Examples	Compiler, assembler, debugger, driver	Word processor, web browser, media player

Operating System functions (memorise this list)

Process management
Scheduling, execution, synchronisation, deadlock handling.

Memory management
Allocation, deallocation, paging, segmentation, virtual memory.

File / disk management
Creation, deletion, organisation, permissions.

Device management
Drivers, interrupt handling, CPU to device communication.

Networking
Communication across connected systems.

Security & protection
Authentication, authorisation, access control.

Plus the **user interface** itself: CLI (command line) or GUI (graphical).

Characteristics: a question can ask for these directly

SYSTEM SOFTWARE

Characteristics

Close to hardware (low-level interaction). Runs in the background, **mostly invisible to users**. Provides services for applications. **Efficient, stable, hardware-dependent**.

APPLICATION SOFTWARE

Characteristics

User-focused (solves problems, boosts productivity, entertains). Runs on top of system software. **General-purpose or domain-specific**. Easy to install / uninstall, user-friendly interface.

The classic classification tree

SYSTEM SOFTWARE BRANCHES INTO

Four types

Operating Systems (Windows, Linux, macOS). **Utility Programs** (Disk Defragmenter, Firewall). **Language Translators**: Assembler, Compiler, Interpreter. **Library Programs**.

APPLICATION SOFTWARE BRANCHES INTO

Three types

General-Purpose Applications (Word Processor, Spreadsheet). **Special-Purpose Applications** (Web Browser, Accounts Management). **User Applications Software**.

10

SOFTWARE

Types of System Programs

These are the specific tools that turn your source code into something the CPU can run.

WRITE THIS **"Explain types of System Programs"**

There are several important types of system programs. A **compiler** translates an entire high-level language program into machine code in one go, like GCC. An **interpreter** executes high-level code line by line instead, like the Python interpreter. An **assembler** converts assembly language code into machine code, like NASM. A **linker** combines multiple object files into a single executable file and resolves references between them, like GNU ld. A **loader** takes this executable file and loads it into main memory so it can run. An **editor**, such as vi or Notepad, is used to write and modify source code. Together, these programs take source code all the way from a text file to a running process in memory.

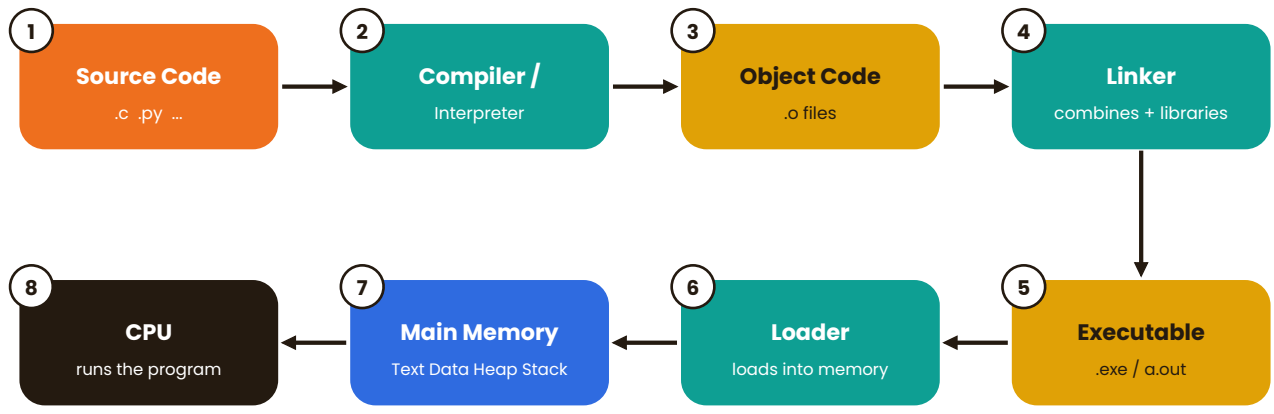


Fig 7: from typed code to a running process (follow the numbers 1 to 8)

PROGRAM	ROLE	EXAMPLE
Compiler	Translates a whole high-level program into machine code, in one pass	GCC, javac
Interpreter	Executes high-level code line by line, no separate machine-code file	Python interpreter, Node.js
Assembler	Converts assembly code into machine code	NASM, MASM
Linker	Combines multiple object files into one executable, resolves symbols	GNU ld
Loader	Loads the executable into main memory to run	OS loader
Editor	Writing / modifying source code or text	vi, nano, Notepad
Translator (general)	Converts code from one language to another	SQL-to-C converters

EASY WAY TO REMEMBER

Compiler = translates the whole book first, then you read it. Interpreter = translates and reads one line at a time.

11

SOFTWARE

Levels of Programming Languages

WRITE THIS **"Explain levels of Programming Languages"**

Programming languages can be divided into three levels. **High-level languages (HLL)**, such as Python, Java, and C#, are closest to human language, are machine-independent, and are easy to read, write, and debug. **Middle-level languages (MLL)**, such as C and C++, act as a bridge: they give the programmer both high-level constructs and direct access to the machine. **Low-level languages (LLL)**, such as Assembly language and machine code, are closest to the hardware, are machine-dependent, and execute very fast, but are much harder to program in.

LEVEL	CHARACTER	EXAMPLES
High-Level (HLL)	Closest to human language, machine-independent, easy to read / write / debug	Python, Java, C#
Middle-Level (MLL)	Bridges HLL and LLL: gives both machine access and high-level constructs	C, C++
Low-Level (LLL)	Closest to hardware, machine-dependent, fast but hard to write	Assembly, Machine code

Same "Hello, World!" at two different levels

HIGH / MIDDLE LEVEL (C)

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

LOW LEVEL (ASSEMBLY)

```
section .data
msg db "Hello, World!", 0Ah
section .text
global _start
_start:
  mov edx, 13    ; message length
  mov ecx, msg   ; message to write
  mov ebx, 1     ; stdout
  mov eax, 4     ; sys_write
  int 0x80       ; call kernel
  mov eax, 1     ; sys_exit
  int 0x80
```

Same output, but very different amounts of control you must spell out yourself. That gap **is the point** of the HLL / MLL / LLL difference.

12 WEB SYSTEMS
Request-Response, Client-Server & 3-Tier

The generic pattern behind almost every web system: a **Requestor** asks something of a **Provider**, and gets a **Response** back. Client-Server is the same pattern with named roles; 3-tier splits the Provider side into three specialised servers.

WRITE THIS **"Explain Request-Response Model / 3-Tier Architecture"**

The **request-response model** is the basic pattern where a requestor sends a request to a provider, and the provider sends back a response. On the web, this is implemented as the **client-server model**, where a client, such as a browser, sends an HTTP request, and a server sends back an HTTP response. In larger systems, this is expanded into a **three-tier architecture**: Tier 1 is the **web server**, which handles the HTTP request and response with the user's browser; Tier 2 is the **application server**, which invokes the required components and processes the logic; and Tier 3 is the **database server**, which executes queries and returns the results back up through the tiers to the user.

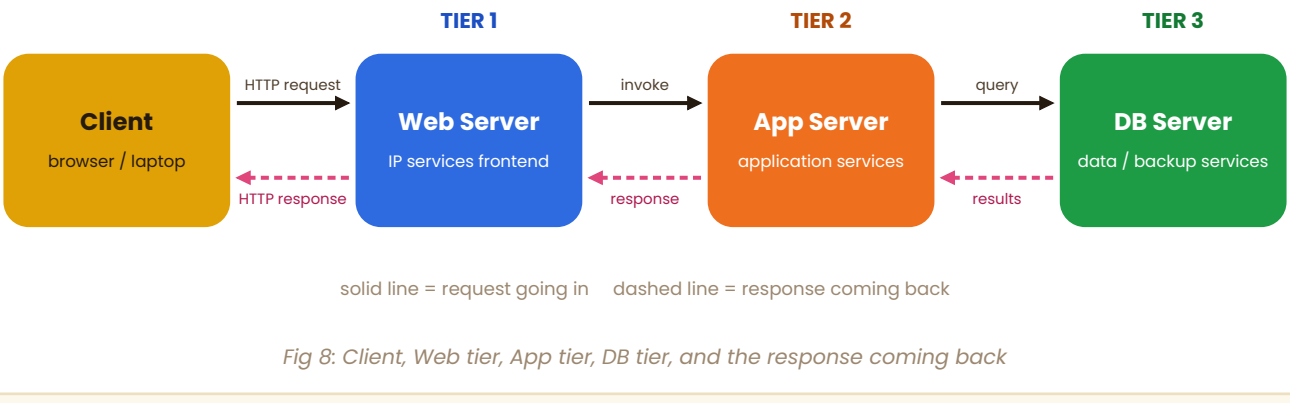


Fig 8: Client, Web tier, App tier, DB tier, and the response coming back

Request-Response
Generic model: Requestor sends a Request, Provider sends back a Response.

Client-Server
Named version: client sends an HTTP request, server sends an HTTP response.

3-Tier
Web Server, then App Server, then Database Server, each with a distinct job.

13 SOFTWARE ENGINEERING

Types of Requirements

WRITE THIS **"Explain types of Requirements"**

Software requirements are of two types. **Functional requirements** describe how the software should actually be used by its users, such as the design of user interface screens, and these are given to the development team by the software or product owner. **Non-functional requirements** describe qualities the software must have, such as **performance, scalability, and security**, and it is the responsibility of the development team to take care of these while designing and developing the software.

TYPE	MEANING	GIVEN BY
Functional	How the software should actually be used by users, e.g. user interface screens	Software / product owner, to the dev team
Non-functional	Performance, scalability, security: the qualities the software must have	The dev team handles these while designing and developing

EXAM ONE-LINER
Functional = **WHAT** it does. Non-functional = **HOW WELL** it does it.

14 SOFTWARE ENGINEERING

Types of Environments

A build travels through a fixed pipeline of environments before it ever reaches a real user.

WRITE THIS **"Explain types of Environments"**

A software build passes through several environments before reaching real users. The **DEV environment** is used by the development team to build and unit-test the software. The **QA environment** is used for system testing, integration testing, and non-functional testing, and needs QA approval before moving forward. The **UAT (User Acceptance Testing) environment** is where the client's own QA team tests both functional and non-functional requirements, and the client's approval is required before going further; performance testing and vulnerability testing can also be done as separate, optional environments at this stage. The **PRODUCTION environment** is the final, live environment where end users actually use the software. Finally, the **Disaster Recovery (DR) environment** is a backup of production, used only if the production environment fails.

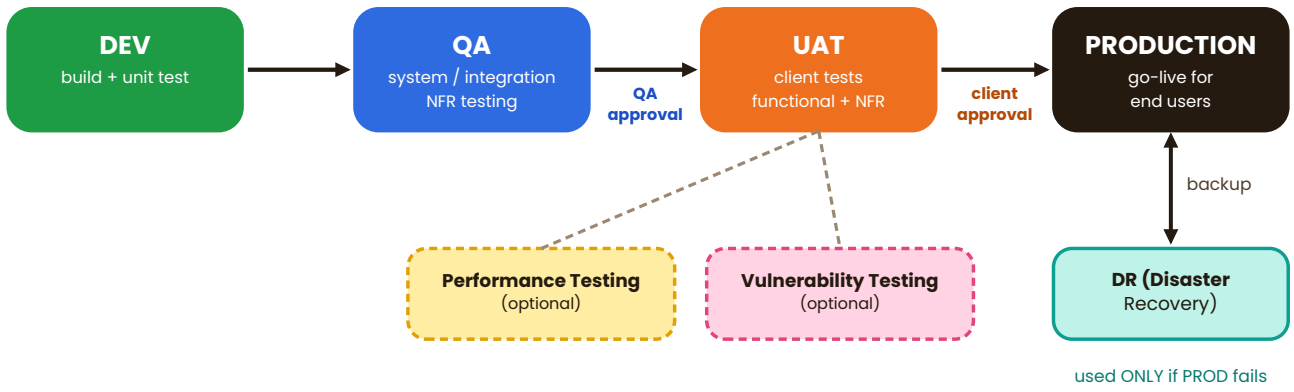


Fig 9: the standard release pipeline

ENVIRONMENT	PRIMARY USE	REMARKS
DEV	Development team builds and unit-tests	-
QA	System, integration and NFR testing	QA approval needed before client UAT
UAT	Client's QA team tests functional + non-functional requirements	Client approval needed before moving to PROD
Performance Testing	Tests the performance NFR specifically	Optional, separate environment
Vulnerability Testing	Tests the security NFR specifically	Optional, separate environment
PRODUCTION	Go-live: end users actually use the software	-
Disaster Recovery (DR)	Backup of PROD	Used only if PROD becomes unavailable

15

LAST-MINUTE REVIEW

Cheat Sheet: every definition in one place

CORE TERMS

Program: instructions on disk, not running.
Process: a program in execution, has a PID.
Thread: lightweight unit inside a process, shares memory, own stack.

CPU INTERNALS

Registers: PC, IR, ACC. Fastest storage, inside CPU.
ALU: arithmetic and logic operations.
Control Unit: decodes instructions, directs everything.
Buses: Data / Address / Control.

INSTRUCTION CYCLE

Fetch, Decode, Execute, Store. Repeats for every instruction; the PC moves on each time.

MEMORY HIERARCHY (FAST TO SLOW)

Registers, Cache, RAM, Secondary, Tertiary. Going down: speed falls, size grows, cost falls.

I/O TECHNIQUES

Programmed I/O: CPU manages it (busy-wait).
Interrupt-driven I/O: device interrupts CPU when ready.
DMA: bypasses the CPU.

SOFTWARE SPLIT

System software: interface between apps and hardware. Low-level, no UI. E.g. OS, compiler.
Application software: user-facing, high-level. E.g. Word, browser.

SYSTEM PROGRAMS

Compiler: whole program to machine code.
Interpreter: line by line.
Assembler: assembly to machine code.
Linker: combines object files. **Loader**: loads into memory.

LANGUAGE LEVELS

HLL: Python, Java, C#. **MLL**: C, C++. **LLL**: Assembly, machine code. From readable to hardware-close.

PROCESS: WITH VS WITHOUT OS

With OS: loader, PCB, scheduler, context switch, OS handles I/O.
Without OS: one program, no scheduling, program handles I/O itself, CPU halts at the end.

WEB ARCHITECTURE

Request-Response: Requestor and Provider.
3-Tier: Web server, App server, DB server.

REQUIREMENTS

Functional: what it does, from the product owner.
Non-functional: performance, scalability, security. The dev team's job.

ENVIRONMENTS

DEV, QA, UAT, PROD, plus optional Performance / Vulnerability testing, plus **DR** as the backup of PROD.

Made from your six uploaded files. Good luck with the mid-sem, you have got this!